



TECHNISCHE HOCHSCHULE NÜRNBERG
GEORG SIMON OHM

Fakultät Elektrotechnik Feinwerktechnik Informatik

Conceptual design and realization of a graphical tool for determining the test coverage of digital circuit diagrams

Masterarbeit im Studiengang Softwareengineering und
Informationstechnik

vorgelegt von

Razvan Matis

Matrikelnummer 2403005

Erstgutachter: Prof. Dr. Flaviu Popp-Nowak

Zweitgutachter: Prof. Dr. Claus Kuntzsch

© 2021

Dieses Werk einschließlic seiner Teile ist **urheberrechtlich geschützt**. Jede Verwertung auSSerhalb der engen Grenzen des Urheberrechtgesetzes ist ohne Zustimmung des Autors unzulässig und strafbar. Das gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmungen sowie die Einspeicherung und Verarbeitung in elektronischen Systemen.

Abstract

According to Printed circuit board (PCB)s and their digital circuits which are getting more complex all the time there is the requirement of being able to determine the test coverage of it. In dependency on these determined values then automatic boundary scan chains could be performed or the need of changing something on the layout will be visible. This document describes the way of the conceptual design and the realization of a graphical tool for loading, showing different digital circuit diagrams projects which need to be in the Open database (ODB)++ format and the determination of test coverage parameters as well as the creation of Serial vector format (SVF) files for being able to run the boundary scan chain on these special devices which contains at least one Joint test action group (JTAG) device. Furthermore there will be described detailed information about the frameworks, programming patterns and best practices which were used for creating this Model-view-viewModel (MVVM) C# project and the project structure itself which was created for making sure to have the following key features available:

- Opening any ODB++ project by connecting to a stand alone Google remote procedure calls (GRPC) server and using the PCB-Investigator API
- Exporting any opened projects to Java script object notation (JSON) file for later usage and importing of that created files
- Creation and loading of project files which contains a manifest file for information about the project structure itself
- Modifying, exporting and importing of different kind of settings according the connection to the GRPC server, the appearance and the test coverage values
- Determination of different test coverage values according to the boundary scan mechanism
- Creation of SVF files for run the boundary scan chain with a programmer from ProMik GmbH on a JTAG containing device

Contents

Abstract	ii
1. List of abbreviations	1
2. Introduction	2
2.1. Motivation	2
2.2. Requirements	2
2.3. Programming language and operating system	4
3. Fundamentals	5
3.1. ODB++ project format	5
3.2. PCB-Investigator	6
3.3. JTAG interface	7
3.4. Boundary scan	9
3.5. Boundary scan description language	10
3.6. SVF file format	11
3.7. Model-view-viewModel	11
3.8. PRISM framework	12
3.9. Dependency injection framework	13
3.10. Material design framework	13
3.11. JSON format	13
3.12. GRPC	14
4. Conceptual design	16
4.1. Programming language to use	16
4.2. Integrated development environment to choose	16
4.3. Application structure to consider	17
4.4. Target frameworks to use	18
4.5. Analyzing the structure of PCB-Investigator API	19
4.6. Converted class structures of objects of interest	20
4.7. Frameworks and patterns to use	21
5. Architecture	23
5.1. Overview	23
5.2. Interfaces	25

5.3. Grpc Server for using the PCB Investigator API	26
5.4. Grpc Client for connecting to the Server and parsing all objects	27
5.5. GUI	29
5.5.1. MainView and MainViewModel	30
5.5.2. Events	31
5.5.3. Helper	32
5.5.4. Interfaces	33
5.5.5. Implementations	35
5.5.6. Model	36
5.5.7. ViewModels	37
5.5.8. Views	38
5.6. Test coverage determination of loaded project	40
5.7. Pin information extraction	43
5.8. SVF file generator	44
5.9. Unit tests	47
6. Analysis	50
6.1. Performance of loading ODB++ projects by using the PCBInvestigator API	50
6.2. Performance of loading projects by importing JSON formatted files	50
6.3. Performance of exporting projects into JSON format	50
6.4. Performance of showing connections of components	50
6.5. Performance of determination of test coverage values	50
7. Conclusion	51
8. Thanksgiving	52
A. Appendix	54
List of Figures	55
List of Tables	56
List of Listings	57
Bibliography	58

Chapter 1.

List of abbreviations

etc. et cetera

PCB Printed circuit board

ODB Open database

JTAG Joint test action group

SVF Serial vector format

GRPC Google remote procedure calls

JSON Java script object notation

MVVM Model-view-viewModel

MCU Micro controller

GUI Graphical user interface

CPU Central processing unit

RAM Random access memory

GB Gigabyte

CAD Computer-aided design

API Application interface

BSDL Boundary scan description language

WPF Windows presentation foundation

XAML Extensible application markup language

IDE Integrated development environment

Chapter 2.

Introduction

Within this chapter the motivation will be described, the requirements, the used programming language and operating system for creation of the whole project. All these aspects were previously discussed with the company to make sure to create a most valuable software for them.

2.1. Motivation

As the company ProMik GmbH is already providing programmers for being able to write data into different Micro controller (**MCU**)s now there came up the requirement, in dependency on customer demands, to perform boundary scan test chains on **JTAG** devices. For being able to perform these kind of tests there should be a software available which allows to determine first the test coverage parameters in dependency on the boundary scan mechanism and create such **SVF** files for playing them with the programmer. This software should be provided as a complex framework where the opportunity is given to extend it in the future in elementary way by using much interfaces within the **MVVM** pattern.

2.2. Requirements

In order to provide all features as needed there were a couple of requirements which popped up at the very beginning of the conceptual design and during the implementation of the software:

- There should be the possibility of opening **ODB++** project folders and display the result of circuit diagram in a useful way
 - There should be the possibility of moving the visible area by mouse movement and zooming in and out of the visible area

- The visibility of each component should be settable
 - All connections of one component to the others should be displayable
 - The whole view of all components could be mirrorable along both axes
 - The net names should be settable to visible and hidden
- An export mechanism of an opened project into **JSON** file for later usage should be available as an according import mechanism to be independent from the **PCB-Investigator** API as this requires a separate license for usage
- There should be the option of creating and loading zip compressed project files containing all necessary information
 - If the project file mode is being used then all exports and opening of components should be done and saved within that project file
 - If the project file contains settings, a **JSON** exported project then these files should be imported automatically on loading
- Settings should be available, exportable and importable
 - General settings according the appearance of the loaded project view
 - Server related connection settings
 - General settings according the request of **PCB-Investigator** API
 - Test coverage related settings for determining all components for the boundary scan chain
 - **SVF** related settings according the programmer connection parameters to use
- Determination of test coverage parameters using the boundary scan mechanism
- Export of a file containing all pin information of all **JTAG** devices
- Creation of **SVF** files containing all instructions to be performed by the programmer while playing the boundary scan test chain on a **JTAG** compatible device
- In order to create an extended-friendly framework there was furthermore the need of usage of company internal libraries instead of creating complete new ones
 - The usage of log panel was required
 - The usage of setting panel was required
 - The usage of **SVF** player was required

2.3. Programming language and operating system

The whole project was implemented with the software development environment called **Microsoft Visual Studio 2019** and the programming language **C#**. This is a quit powerful and modern language for creation of different desktop applications. The developer environment is really comfortable too, as it provides a NuGet package manager for organizing all external included libraries. Furthermore the whole project was created on a operating system **Windows 10 Professional x64** using a Central processing unit (**CPU**) of type **Intel(R)Xeon(R) E3-1240 v6 @ 3.70 GHz** and a total amount of Random access memory (**RAM**) of **16 Gigabyte (GB)**. This system makes sure to be able to implement without any delays of waiting for background processes at all and ensures to be able to run the developed software almost immediately without any delay times.

Chapter 3.

Fundamentals

Within this chapter the fundamentals regarding the later usage for implementation of the project is being described.

3.1. ODB++ project format

The ODB++ format is a de facto standard format for creation of Computer-aided design (CAD) related projects[1] within the circuit board industry. This format itself was released in 1995 and after the component names were added then the suffix ++ was added in 1997. It hierarchically contains all necessary information about the digital circuit diagram in detail:

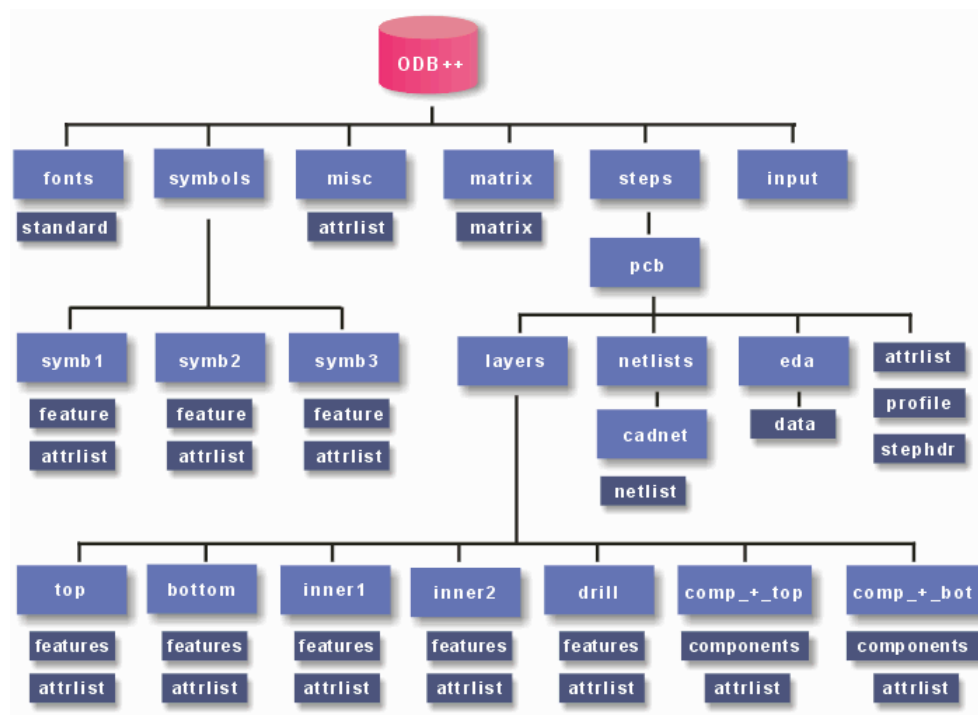


Figure 3.1.: ODB++ project structure[2]

As shown in 3.1 it is clear that this format contains all necessary components, net lists and attribute lists which are necessary for later determination of the test coverage of the circuit diagrams. These data will give detailed information regarding the placed components, their names, positions, sizes and connection to all nets. In dependency on the net connection the connected components could then be determined

3.2. PCB-Investigator

The **PCB-Investigator** is a software provided by EasyLogix also known as Schindler & Schill GmbH and provides many functionality regarding the opening, viewing and measuring of different kind of **CAD** related project files of digital circuit diagrams[3]:

- **ODB++**
- Gerber 274x
- Gen Cad 1.4
- Eagle files
- IDF files
- DXF files
- DPF files
- IPC2581 files
- IPC356 files
- PDF

It is a quit powerful tool which further provides the possibility to export almost all project types into another one and generate different kind of lists containing component relevant information as the component name and its pin information. Further the executable file application itself provides different public functions which are accessible by adding the **PCB-Investigator.exe** as a reference to the project.

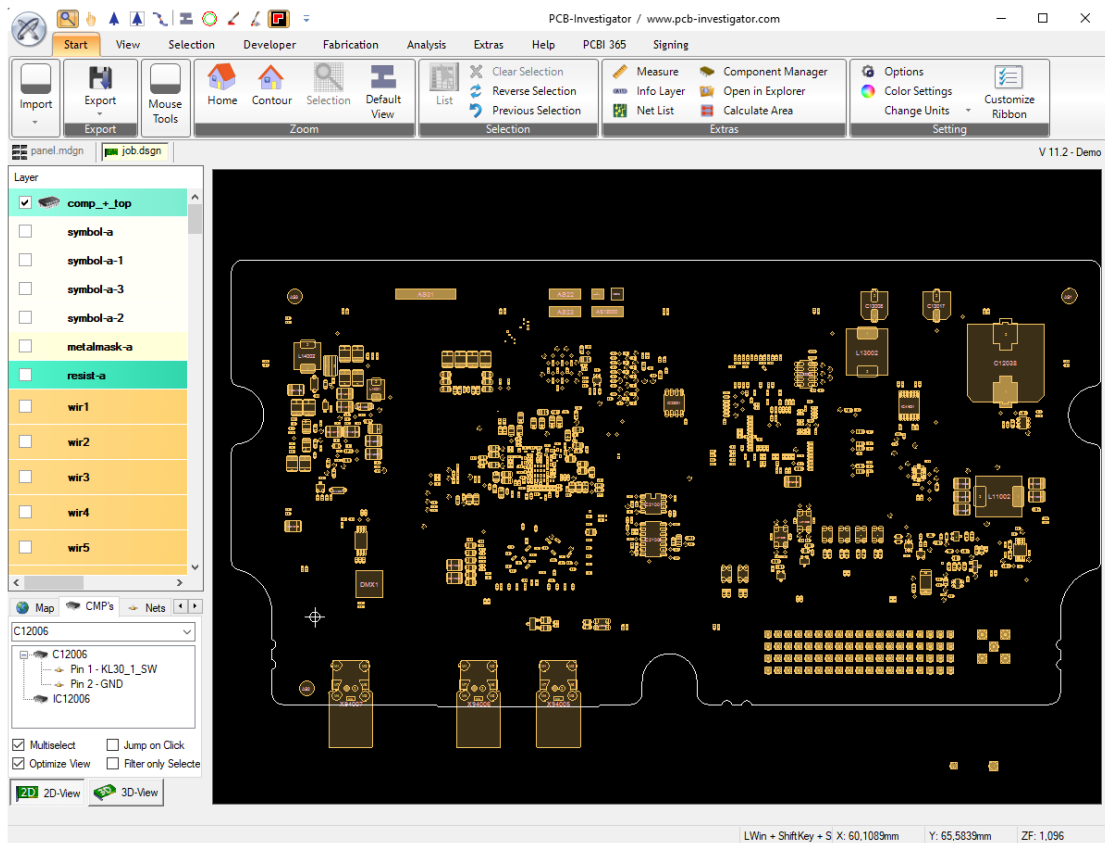


Figure 3.2.: PCB-Investigator overview

As pictured in 3.2 it can be seen that there are many possibilities of taking action with a loaded project. For verification purposes the most important feature was the CMPs tab where it is possible to search for any specific component in dependency on its name and to have a closer look in its consisting pins.

3.3. JTAG interface

The **JTAG** is a serial interface which is widely used for different field programmable gate arrays, complex programmable logic device configurations, the programming and debugging of different micro controllers. It is usually operating within the voltage range of 1.8V - 6V and consists of the following components in general:

- Test access port
 - Pin TDI - data in

- Pin TDO - data out
 - Pin TMS - control
 - Pin TCK - clock
 - Pin TRST - test reset
- Instruction register
 - Data register
 - TAP controller

This standard established within 1985/1986 and is being normed by IEEE 1149.11990. Later on by the overworked norm IEEE 1149.11994 the Boundary scan description language (**BSDL**) was considered as a default feature. The most actual version of the norm is IEEE 1149.1-2001. If there are more TAPs connected together then it will be called a scan chain in general.

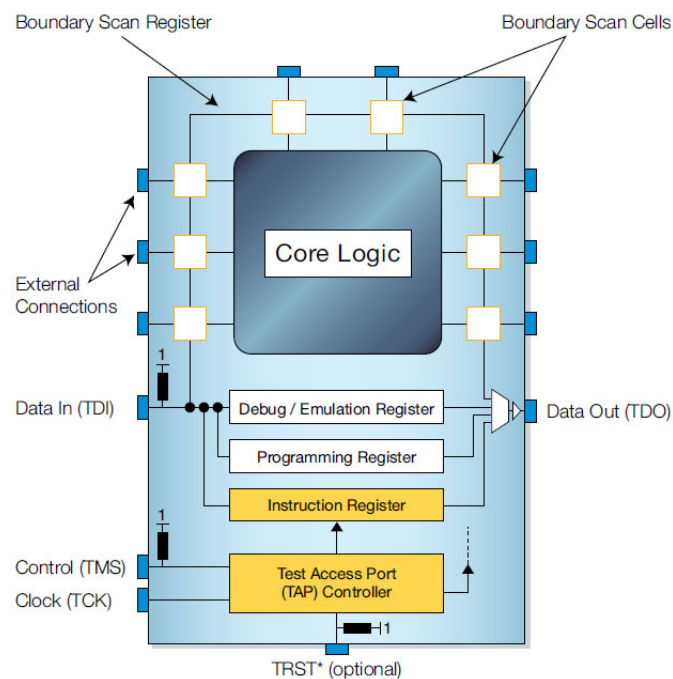


Figure 3.3.: JTAG interface in detail[4]

As shown in 3.3 it can be seen that that the **JTAG** device itself already contains some addition core logic for being able to perform different kind of test-debugging functions. The boundary scan register itself contains a various amount of boundary scan cells which could be used for the testing logic.

The TAP controller is a state machine which could take in six valid different states[5].

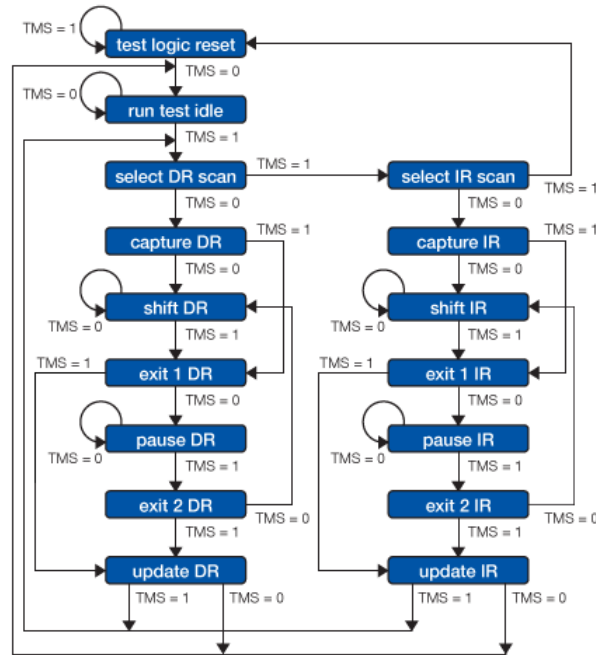


Figure 3.4.: State machine of the TAP controller[6]

As shown in 3.4 it will be clear that the transitions of each state are being controlled mainly by the TMS signal. Every state has two outgoing transitions to make sure that just by one TMS step the condition could be set. The both main paths allow reading out data from the data register or the instruction register.

3.4. Boundary scan

The main idea of the boundary scan is the purpose of ensuring that every component is mounted correctly in physically way as described within the circuit diagram[7]. First of all according to IEEE 1149.1 an **JTAG** device needs to have the possibility of performing the following instructions to be a really boundary scan compatible device[6]:

- **BYPASS** - this instruction ensures that the TDI and TDO connections are connected by a one bit register and makes possible to save time during the operation
- **EXTEST** - this instruction makes sure that the TDI and TDO are connected with the boundary scan registers by providing functions to write values into the cells

- **SAMPLE/PRELOAD** - this instruction ensures that the TDI and TDO are connected with the boundary scan while operating in usual mode. within this mode the data could be read out

In detail that means that according to each external pin connection there is in general the possibility to set any value to high or low and read back the physically present value of wiring. So if it is known for example that on a specific pin position an according pull down resistor is being connected then the further knowledge is that in every case the read back value should be low. Additionally there can be assumed that if a pull up resistor is being connected to a pin independently on the set value the read back value should be high. Finally the fact can be accepted that if some other components are connected to any pin which are not belonging to the pull down nor the pull up resistors then the read back value should be that one which was previously set. To perform a complete test of a device the following steps must be followed:

1. The clock rate must be triggered as long as the complete boundary scan register length totally is[8]
2. During that time the TDI data will be forwarded into each boundary scan cell
3. If all cells are covered by a value then the TMS pin should be triggered for passing all read values into the connected pins
4. The clock rate must be triggered again for the total length of the boundary scan register
5. During that time all values from the external pins are being read out and transmitted into the TDO
6. Now the read out values could be compared with some expected values to detect differences

3.5. Boundary scan description language

The boundary scan description language is a language for describing the boundary scan test ability of different **JTAG** compatible devices. It is based on the very high speed integrated circuit hardware description language and normed with IEEE 1149.1-2001. Typically these information are being provided by the manufacturer of the devices in a .bsdl file for each component[9]. The files themselves are usually containing the following sections:

- **port** - the information about all pins and their usage mode (in, out, inout, linkage...)

- attribute PIN_MAP - the mapping from the used names within the port section to the real pin names used within the **ODB++** project (position is important for later usage)
- attribute INSTRUCTION_LENGTH - instruction register length
- attribute BOUNDARY_REGISTER - all boundary scan compatible pins and their functionality (control, bidirectional, observe_only, output2)
- attribute BOUNDARY_LENGTH - the amount of boundary scan cells

3.6. SVF file format

The serial vector format is a format for communication with boundary scan compatible test vectors. It was introduced by Texas Instruments in 1991. The main purpose of the format is to ensure that independently on the used **JTAG** device different test operations could be performed successfully[10]. The format contains a couple of different instructions which could be performed by the boundary scan logic. The most important one according to this project is the **SDR length [TDI (tdi)] [TDO (tdo)][MASK (mask)] [SMASK (smask)];**. SDR means scan data register and tells the test to test all data register cells. The TDI is the data which should be passed into the registers[11]. TDO is the expected data to be read back from the data registers. And the MASK is the information about the positions of interest which should be considered. A valid instruction example is

SDR 24 TDI(000010) TDO(818181) MASK(FFFFFF);

In that specific example we have a total length of boundary scan cells of 24 and therefore the amount of passed hexadecimal coded byte values is 3 (24 bits / 8 bits/byte) -> 0x00 0x10 and 0x81 0x81 0x81 and 0xFF 0xFF 0xFF. Further the TDI value 000010₁₆ means that only on position 5 the value should be set to high (10₁₆ -> 00010000₂, counted from right to left). The TDO value 818181₁₆ gives the information about the expected positions to be high after read back the data (818181₁₆ -> 100000011000000110000001₂). That signifies only the positions 1, 8, 9, 16, 17 and 24 (read from right to left) should be set to high as the needed result. Finally the MASK value FFFFFFF₁₆ shows that all positions of the 24 bit register should be considered.

3.7. Model-view-viewModel

The Model-view-viewModel pattern came up with the introduction of the Windows presentation foundation (**WPF**) framework from Microsoft and is designed for loose coupling

programming with different components which are working together at the final stage. It could be seen as an enhancement of the model view controller pattern from JAVA programming language [12].

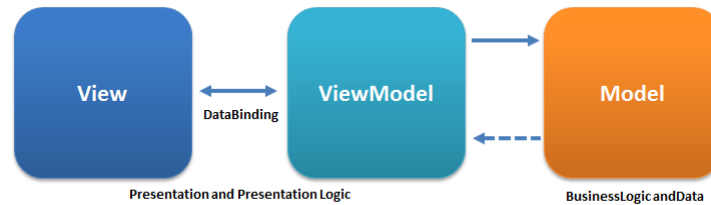


Figure 3.5.: MVVM structure[13]

As shown in 3.5 there are the three main components involved:

- View - the graphical representation form typically created in Extensible application markup language (XAML) code
- Model - content of all data to be used and business logic for it
- ViewModel - the connection layer to the view and contains representation logic

Typically there will be defined a various amount of data bindings in the view to the viewmodel. Further the view model itself informs the view about happened changes of values to be displayed by using the `INotifyPropertyChanged` event. Onward it is responsible for the connection to the data of any model and controls the function sequence to be performed in dependency on the user inputs which are being forwarded as `ICommand` objects. Finally the view should contain as less behind code as possible to make sure a high loose coupling is being ensured. The looser the coupling the easier it is to switch between new implementations of different components and replace them by others. In that way it should be quit easy possible to work on all three component types separately, without having to know of each others and finally connect them together.

3.8. PRISM framework

This framework is made for creation of louse coupled **MVVM** projects and allows to use predefined services as `EventAggregator`, the composite commands and dependency injection in a quit easy way[14]. Further it provides different project templates which could be used as start point of implementation.

3.9. Dependency injection framework

The **dependency injection framework** provides the possibility of registering interfaces by their implementations at some main points and being able to inject them directly in constructors of others classes without managing the creation of objects manually in background[15]. It can use further the **singleton pattern** for making sure just to work with a single instance of a class project wide.

3.10. Material design framework

The material design framework is a view orientated framework where the options are given to use many predefined components in a pretty and comfortable way[16]. Further it ensures a quit modern look and feel and will be used per default in many actual software implementations as in web design.

3.11. JSON format

The java script object notation is a serialized form of data output in textual form[17]. It is widely used where it could be necessary to understand and maybe modify data manually in an easy way. To work with that specific data format each programming language provides different kind of external libraries for parsing the information from objects into **JSON** and back from **JSON** to object. A valid **JSON** formatted text could like like:

```
1 {
2   "Pins": [
3     {
4       "Nets": [
5         0
6       ],
7       "Geometrics": {
8         "Bounds": {
9           "X": -1459,
10          "Y": 286,
11        },
12        "UseRotation": true
13      },
14      "PinType": 2,
```

```

15         "PinNumber": "Pin1"
16     }
17 ]
18 }

```

Listing 3.1: JSON formatted content

As shown in 3.1 the format is highly structured and build in hierarchical way. Further it is clear that the curly brackets are being used for defining an object instance, the square brackets are being used for identifying an array of instances and the double quotes are describing a string value. With that quit simple format is is possible to represent just any Boolean, number or string value of objects where the requirement comes up either to use serializable or plain object structures.

3.12. GRPC

GRPC is a modern open source way for performing remote procedure calls independently on the programming language used for a specific program and works as a client-server structure[18]. That means in detail to use this protocol there is the need first of defining a specific service file using the interface definition language. By using this language there is no need to consider programming language specific dependencies. Afterward by using protoc compiler there is the possibility of automated creation of the source code for a specific programming language which could then be included to finalize the communication work. Finally on one side there should be defined the logic for being a server by additionally defining a specific port to be used and on the other there needs to be defined the logic for being the client with considering the IP address of the server and the port. Using **GRPC** ensures bidirectional streaming possibilities and passes all object related data as a auto serialized stream in the **JSON** format. Further it can communicate through complex networks without having the challenge of defining more complex code for it. To define the service file there are many of elementary data types available, the possibility of defining object structures, functions and parameters or return values. A valid example of such a definition could be:

```

1 syntax = "proto3";
2
3 service PcbInvestigatorService {
4     // Gets all objects
5     rpc GetConvertedObjects(Request) returns (Result) {};
6 }
7
8 message Request {
9     int32 requestNumber = 1;

```

```
10   ComponentTypeGrpc type = 2;
11 }
12
13 message Result {
14     bool status = 1;
15 }
16
17 enum ComponentTypeGrpc {
18     UnknownComponentType = 0;
19     Arc = 1;
20 }
```

Listing 3.2: Interface definition of GRPC exchange

As shown in 3.2 there are all necessary values defined in structured and easy understandable way. Typically the auto generated code contains new classes which must be inherited for adding customized logic.

Chapter 4.

Conceptual design

Within this chapter all thoughts and decisions which were made according the completion of the whole project are being pointed out in detail

4.1. Programming language to use

First of all there has been reflected which programming language to use for creating the new software. As there are many different languages available the following table shows the resulting advantages of two compared languages:

Programming language	Benefits	Disadvantages
C#	modern GUI applications possible, powerful, knowledge already available, already being used at ProMik	-
Java Swing	GUI applications possible	no loose coupling available, just little knowledge available, not modern and powerful
Java FX	modern GUI applications possible, powerful	no knowledge available, not being used at all at ProMik

Table 4.1.: Programming languages comparison

As stated in 4.1 it will be clear that the choice of the programming language to use finally turned into C# as the advantages convinced.

4.2. Integrated development environment to choose

Now there should be chosen an Integrated development environment (IDE) for creating the software with the previously concluded programming language as well:

IDE	Benefits	Disadvantages
JetBrains Rider	powerful	not for free available
Visual Studio Code	for free available	no nuget package manager available
Visual Studio 2019	powerful, for free available, already being used within ProMik	-

Table 4.2.: IDE comparison

As compared in 4.2 it sticks out that Visual Studio 2019 will be the choice for creating the new software as it's benefits are overbalanced.

4.3. Application structure to consider

Regarding the application structure there was the deliberation about how to use the PCB-Investigator API as efficient as possible without having the need of installing it on every machine which runs the new tool as there is the requirement of having a valid license for using it. The idea came up to provide on one side a stand alone application which uses the PCB-Investigator API as a server and on the other side the Graphical user interface (GUI) application as client software which is possible to connect to the server. In that case this server could potentially run just on one machine and would be available for all other clients which are in the same network.

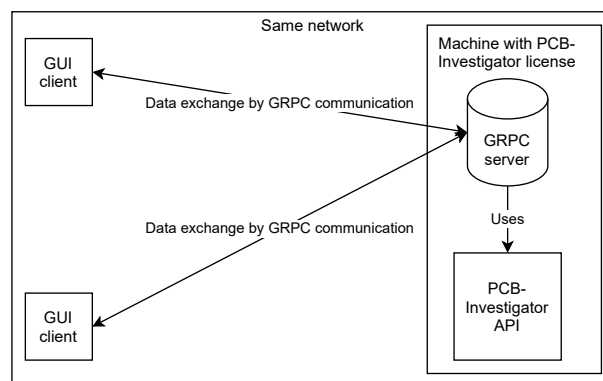


Figure 4.1.: Client-server structure of the planned software

As shown in 4.1 it is clear that following that idea a most flexible solution with as less restrictions as possible could be created. To make sure that the clients are able to communicate with the server the decision was made to use the GRPC protocol for data

exchange as it is already known within the company as a reliable protocol and could be used independently on the used programming languages or target frameworks of the projects.

4.4. Target frameworks to use

After the programming language, the **IDE** are selected and the application structure itself was defined, now there should be decided which target frameworks to use for all projects:

Framework	Benefits	Disadvantages
.NET framework	the only compatible one for using the PCB-Investigator API	not fully compatible with all provided nuget packages, no longer support
.NET standard	compatible to be included in .NET framework and .NET core applications	no graphical components available
.NET core	all modern graphical components available, all nuget packages compatible	graphical components included even if not used

Table 4.3.: Framework comparison

As shown in 4.3 each available framework has some advantages which makes it worth to use. So after thinking about it more in detail the following project structure came out to use:

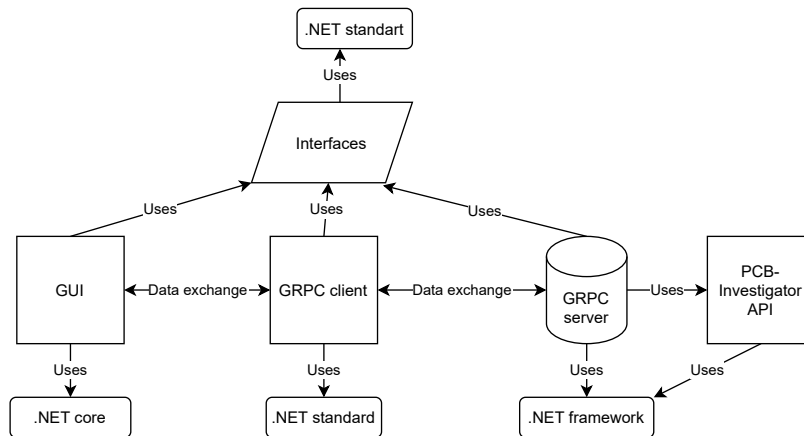


Figure 4.2.: Frameworks used within the project

The reason why to use .NET framework 4.72 for the server side is the fact that the provided API was created with that target framework (see 4.2). On the client side the reflection was considered to create a modern software as possible by using as much

useful frameworks for it as available and so the .NET core 3.1 has been chosen as target framework. To make sure finally that each project are able to use the common interfaces which should be created the target framework .NET standard 2.0 was used for it.

4.5. Analyzing the structure of PCB-Investigator API

Of course the structure of the **PCB-Investigator** API itself regarding the received objects needs to be analyzed more in detail which are being provided by the **PCB-Investigator.exe**:

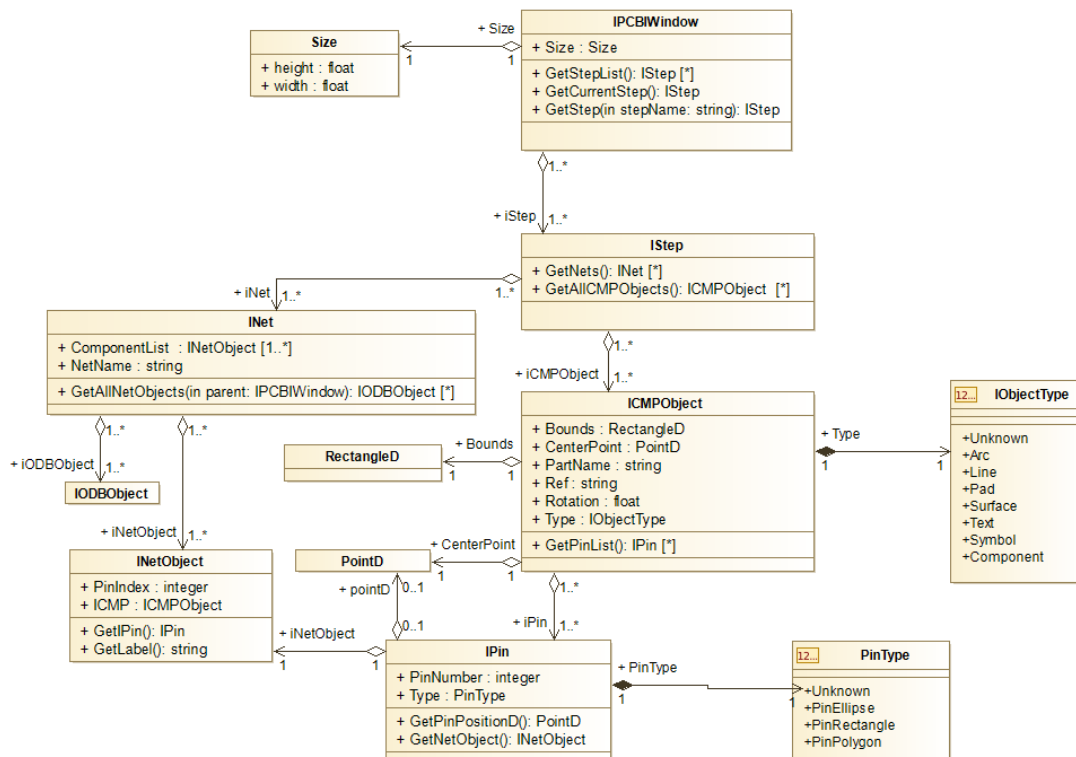


Figure 4.3.: Class diagram of the PCB-Investigator objects of interest

As shown within 4.3 there could be seen that the very first class is the IPCBIWindow which contains a various amount of IStep objects. Within this IStep objects there are many INet objects and ICMPObjets.

As ProMik GmbH is just interested in extracting all relevant information from any **ODB++** related projects the following public functions of the **PCB-Investigator.exe** where used by the created software[19] to extract all INet and ICMPObjets:

- IAutomation.IAutomationInit() - Initialization of the Application interface (**API**)

- `IAutomation.CreateNewPCBIWindow(bool visible)` - Creates the base window
- `IPCBIWindow.LoadData(string path)` - Loads a specific project into the window
- `IPCBIWindow.GetStepList()` - Getter for all steps of the window
- `IStep.GetNets()` - Getter for all nets of a step
- `IStep.GetAllLayerNames()` - Getter for all layer names of a step
- `IStep.GetLayer(string layerName)` - Getter for a layer in dependency on the layer name from a step
- `ILayer.GetAllLayerObjects()` - Getter for all objects which are located on a specific layer within a step

4.6. Converted class structures of objects of interest

After knowing what kind of functions to be used and which structures are available within the **PCB-Investigator** API the goal was to provide as simple new structures as possible containing all relevant information for later usage:

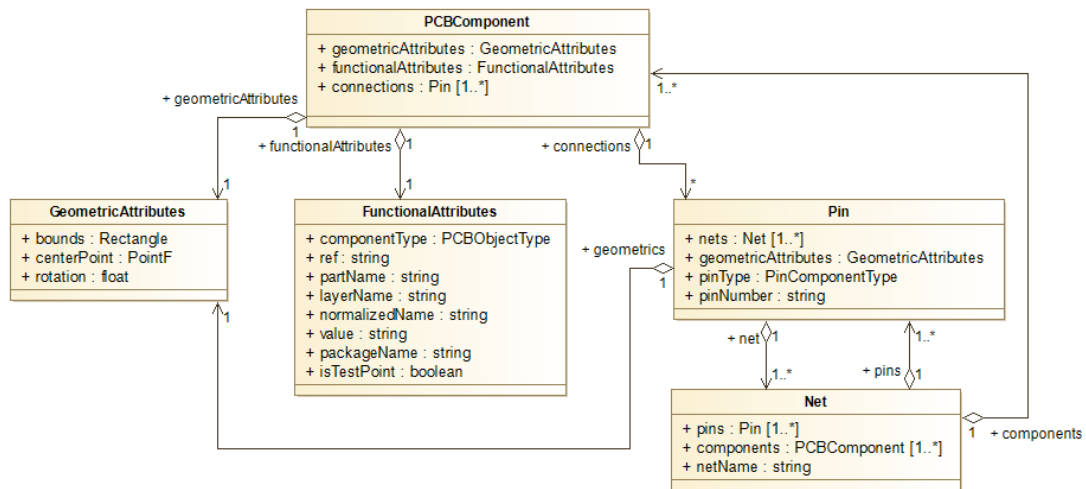


Figure 4.4.: Class structure after conversion

As shown in 4.4 it will be clear that the new object structure just contains five main components

- PCBComponent - the main component of interest containing further objects with information about their geometric, functional attributes and connection pins
- Pin - the connection component of a PCBComponent which contains net objects and information about the pin name and its type
- Net - the net component containing the net name, information about all pins and components which are included
- GeometricAttributes - an object for holding all information regarding the position and size of any object
- FunctionalAttributes - an object for having all information regarding the functionality of one component (type and names)

To make sure for having a highest level on flexibility of course as top level objects there should be interfaces introduced which then will be implemented for later usage.

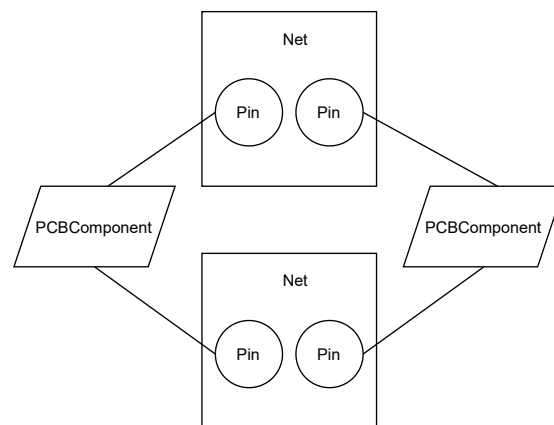


Figure 4.5.: Physical object dependencies

As shown in 4.5 it is possible to map all physical existing objects easily in complete way into the object orientation which is used by the programming language by using this class structure.

4.7. Frameworks and patterns to use

To ensure a modern, loose coupled appearance of the whole application where it is quite easy possible to build in different kind of extensions there was made the decision of using the **PRISM framework** for referencing a template as an empty new project.

Within that framework already the possibility of using the **dependency injection framework** is given which should be used as well to make sure to be flexible in changing the implementations used for different interfaces. Further PRISM makes possible to use the modern appearance style from the **material design framework** which should be used as well to make sure the software looks modern and pretty. To ensure creation of different implementations of interfaces within just one method[20] there was additionally the need of using the **factory pattern** at some points. Next there should be used the **command pattern** to make sure that a loose coupling between views and view-models could be established without any code behind in the views. Finally for being able to work with as many interfaces as possible to ensure easy change of implementations there should be used the **strategy pattern** which exactly means that there are many implementations of interfaces available[21].

Chapter 5.

Architecture

Within this chapter the realized architecture of the implemented software will be described in detail according to the used project structure of the visual studio solution.

5.1. Overview

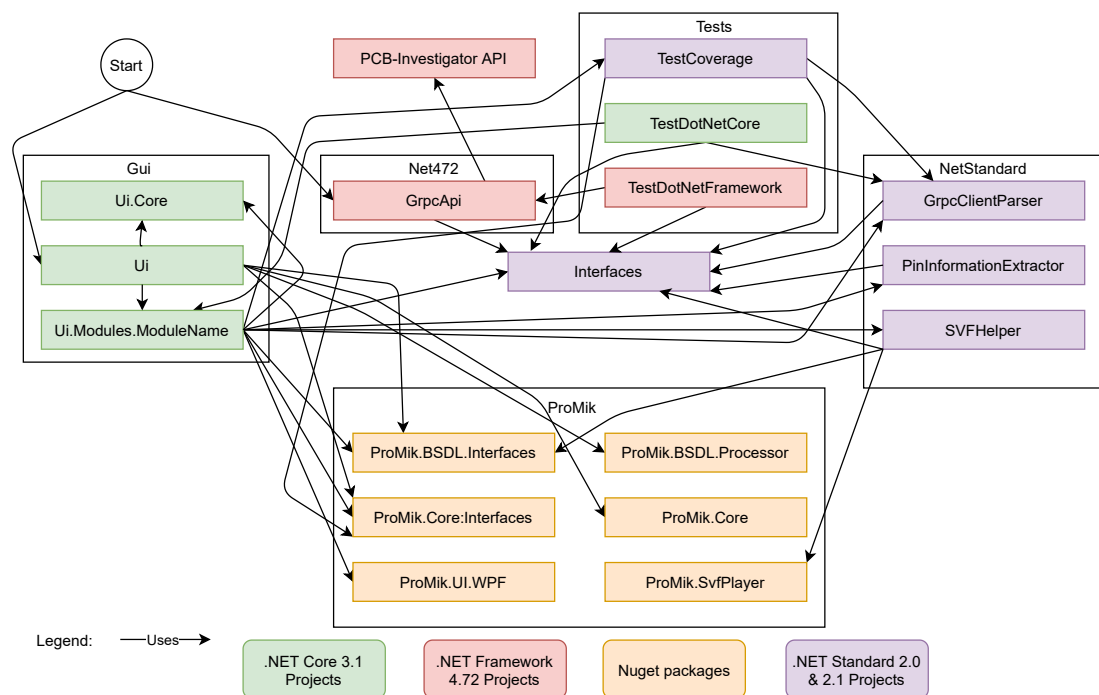


Figure 5.1.: Project overview

As shown in 5.1 it will be clear that the whole project is organized in different kind of folders which contain C# projects. On one side there is the **Gui** folder containing the following projects:

- Ui.Core - This project contains the region names and the view model bases which should be inherited and was provided by the PRISM framework.
- Ui - Within the project the mainView and mainViewModel could be found (Was initially created by the PRISM framework)
- Ui.Modules.ModuleName - This project was basically created by the PRISM framework and contains all necessary events, interfaces, models, viewmodels and views for operating with all other referenced projects

On the other side there is the **Net472** folder with the project GrpcApi which contains the standalone software for referencing the **PCB-Investigator** API and runs as a **GRPC** server. Next the **NetStandard** folder contains the following projects:

- GrpcClientParser - The **GRPC** client project for communicating with the GrpcApi project for getting all parsed objects by the **PCB-Investigator** API
- PinInformationExtractor - This project is being used for creation of textfiles in **JSON** format containing all pin information of all **JTAG** devices of one project.
- SVFHelper - This project is needed for creation of **SVF** files in dependency on **BSDL** files which should be referenced for **JTAG** devices. Further it could play **SVF** files with a programmer from ProMik which is connected to a boundary scan compatible **JTAG** device

The **Tests** folder consists of the following projects:

- TestCoverage - This project contains all boundary scan test according logic to be performed
- TestDotNetCore - Here there can be found different unit tests for the parser logic of the GrpcClientParser project
- TestDotNetFramework - This project contains unit tests to be performed using the GrpcApi project

Finally there were different **ProMik** related projects included by the nuget package manager of visual studio:

- ProMik.BSDL.Interfaces - Contains all interfaces regarding the BSDL handling
- ProMik.BSDL-Processor - Contains the implementations regarding the BSDL processor
- ProMik.Core.Interfaces - The interfaces of core logic
- ProMik.Core - The implementations of core logic

- ProMik.UI.WPF - Contains different Ui elements which could be included (settings panel, logPanel et cetera (**etc.**))
- ProMik.SvfPlayer - Contains the logic for creation and playing of **SVF** files

5.2. Interfaces

The Interfaces project was created as a .NET standart 2.0 project to make sure it could be referenced by any other projects which needs it. It has the following content:

- Gui - Within this folder are just interfaces located which are related to the Gui logic
 - IDialogSelector - It could be used for the handling of dialogs which should be shown either for file opening, saving or folder opening or saving
 - ILogger - The system wide used interface for logging purposes which additionally will be written into a local logfile
- Helper - A folder wich contains objects which were necessary for the two unit test related projects
 - PcbTestObject - This is a helper class for unit test purposes
 - PinTestObject - It is a helper class which is been included in the PcbTestObjects class and provides more detailed pin information
- PcbInvestigator - That folder contains all **PCB**-Investigator related interfaces and their implementations of the received objects
 - IFunctionalAttributes - The interface which contains all functional related attributes
 - IGeometricAttributes - A interface which contains all geometric and position related information
 - INetComponent - This is the interface for representing a net object
 - IPinComponent - That is a interface for describing a pin object
 - IPCBComponent - An interface which contains all information about a PCB-Component
 - Enums - Within that folder are the enumerations placed which were used within the IPCBComponent and IPinComponent for defining the componentType and pinType

- Implementations - In that folder are all implementations of the predefined interfaces of the PCBInvestigator objects located

5.3. Grpc Server for using the PCB Investigator API

This project is responsible for using the **PCB**-Investigator API which is being created using the .NET Framework and was created as a console application using the .NET Framework 4.72 as well. It further derives a service which was provided by a **GRPC** auto code generation as a server. The proto file itself which was used for the code generation contains three functions:

- GetConvertedObjects(Request) returns (Result) - Within this function all objects of a specific project could be determined using the location to the project
- GetConvertedObjectsByZip(RequestZip) returns (Result) - That method obtains all objects by using the received project as zip file in bytes
- ClientAvailable(Empty) returns (StatusResult) - This function is being used to detect whether the client has a valid connection to the server using the settings or not

To make sure that the server itself knows which data it has to open actually only the GetConvertedObjectsByZip method is being used where the project to be opened will be transferred as a byte array containing the a single zip file.

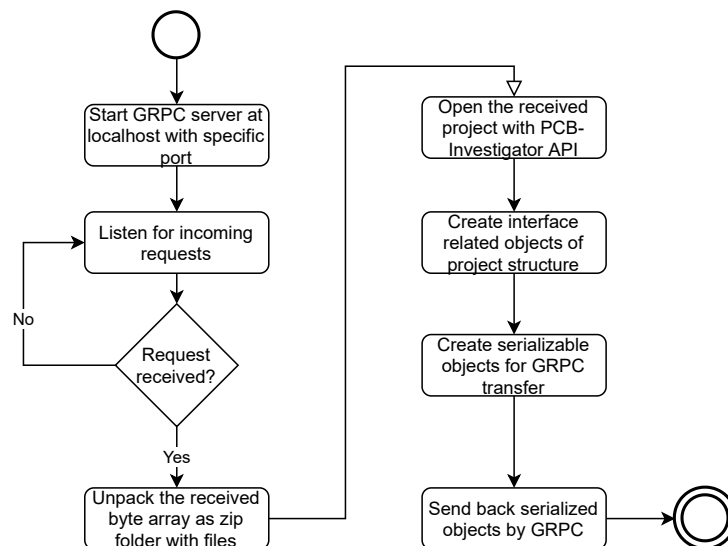


Figure 5.2.: Program schedule of the Grpc server project

As shown in 5.2 it will be clear that the program schedule ends with the serialization of all objects. To be able to do that in background there were created a couple of lists containing each new element to be saved. Finally only the index from the list according an object is being used for referencing it.

```

1      public IList<PinGrpc> GetPinsGrpc(IList<IPinComponent>
      allPins, IList<INetComponent> allNets)
2      {
3          List<PinGrpc> pins = new List<PinGrpc>();
4          foreach (IPinComponent pin in allPins)
5          {
6              IList<int> nets = GetNetIdsGrpc(pin.Nets, allNets);
7              PinGrpc pinGrpc = new PinGrpc()
8              {
9                  Geometrics =
10                     GetGeometricsGrpc(pin.GeometricAttributes),
11                     PinType = GetPinTypeGrpc(pin.PinType),
12                     PinNumber = pin.PinNumber,
13             };
14             pinGrpc.Nets.AddRange(nets);
15             pins.Add(pinGrpc);
16         }
17
18         return pins;
19     }

```

Listing 5.1: GetPinsGrpc

As described in 5.1 there could be seen that for creation of the serialized objects there were a couple of helper methods used to ensure a structured way of working. Further it will be clear that working with a list containing just int objects for referencing the net objects was necessary.

5.4. Grpc Client for connecting to the Server and parsing all objects

For being able to request all objects within a ODB++ project there is the GrpcClient-Parser project available which was introduced as a .NET standard 2.1 project for making sure to be able to include it in as many other projects as possible. That means in detail that this whole project, which is being provided as a GRPC client project, could also be included just as a dynamic link library file within other projects to be able to handle the requests in most flexible way.

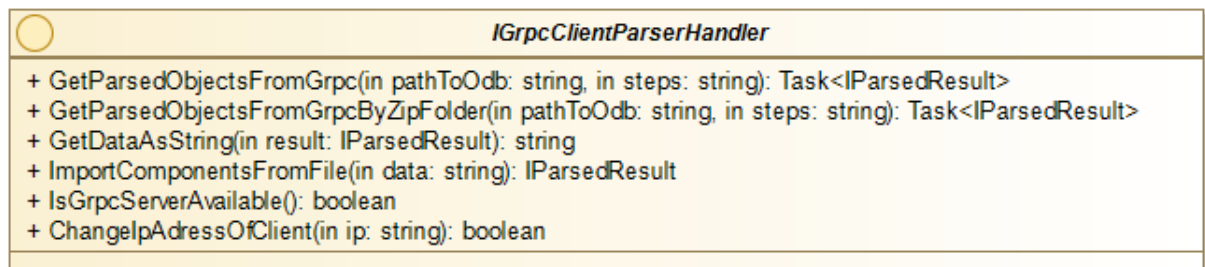


Figure 5.3.: IGrpcClientParserHandler class diagram

As shown in 5.3 the main interface of this project provides functions for being able to request all objects just by passing one path, by passing one path and using its zip content as well as the functionality of being able to import data from a JSON formatted file. Further it has some functions regarding the connection state of the server availability. The IParsedResult interface is a model holding all necessary data of interests. It contains all pins, nets and components of one project.

```

1      public static IPCBComponent GetComponentByType(
2          IGeometricAttributes geometricAttributes,
3          IFunctionalAttributes functionalAttributes,
4          IList<IPinComponent> connections,
5          IList<string> rDef = null,
6          ...)
7      {
8          ...
9          string beginPartName =
10             functionalAttributes.PartName.ToLower(System.Globalization.
11             CultureInfo.CurrentCulture);
12          string beginRef =
13             functionalAttributes.Ref.ToLower(System.Globalization.CultureInfo.
14             CultureInfo.CurrentCulture);
15          if (!useContains)
16          {
17              if (MatchesType(rDef, beginPartName, beginRef, false))
18              {
19                  return new PCBResistor(geometricAttributes,
20                      functionalAttributes, connections);
21              }
22              else if (MatchesType(cDef, beginPartName, beginRef,
23                  false))
24              {

```



```

21         return new PCBCapacitor(geometricAttributes,
22                                 functionalAttributes, connections);
23     }
24     else if (MatchesType(iDef, beginPartName, beginRef,
25                         false))
26     {
27         return new PCBInduction(geometricAttributes,
28                                 functionalAttributes, connections);
29     }
30     else if (MatchesType(icDef, beginPartName, beginRef,
31                         false))
32     {
33         return new PCBic(geometricAttributes,
34                           functionalAttributes, connections);
35     }
36     ...

```

Listing 5.2: GetComponentByType

As stated within 5.2 this is one of the main **factory pattern** using functions as there will be generated different kind of implementations of one interface in dependency on input variables passed which are defining the object identifier. Further there is the possibility provided of using a CsvReader helper class for being able to read in comma separated values files which contain all values of the objects of one project in dependency on their **Ref** attribute.

5.5. GUI

Within this section the **Gui** folder and its project related contents which are all implemented using the .NET Core 3.1 framework will be described more in detail.

5.5.1. MainView and MainViewModel

The mainView and mainViewModel are responsible for creation of the view to be started and showed to the user. It consists of many regions which could be used by the PRISM framework for dynamically assignment of views during the run time:

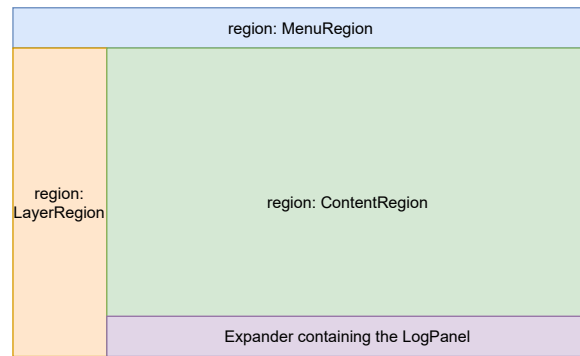


Figure 5.4.: The mainView with its regions

As shown in 5.4 there are some regions defined which are implemented at another project place and will be included dynamically during the software life cycle for having the maximum amount of loose coupling and the opportunity of changing templates to be used. The detailed content of each region are:

- **MenuRegion** - contains all menu items which could be clicked by the user for interaction with the software (importing, exporting of data, showing all settings, *etc.*)
- **LayerRegion** - this region contains on one side all received layers and its objects which can be shown by the **PCB**-Investigator API and on the other side all test coverage related object types which could be selected to be shown (pull down resistors, ics, pull up resistors, *etc.*)
- **ContentRegion** - that region contains all objects which should be displayed after there were some layers or object types selected within the LayerRegion. Further it provides the possibility of component specific actions by the context menu of the right mouse (toggle visibility, show connections, *etc.*)
- **Expander containing the LogPanel** - here the Expander object is being used which contains the ProMik.UI.WPF.LogPanel object by the nuget package manager for displaying all logged messages of the whole application considering each log category for defining the display style

5.5.2. Events

For being able to interact project wide with different classes within different projects without having the need always to reference directly a object of class of interest there was used a general event handling by using the `ProMik.Core.Services.EventService.PrismEventService` which is based on the PRISM event service as a nuget referenced package. In general the event handling works on one side by subscribing to any event type and on the other by publishing that specific event. To specify an event which could use by this service it was necessary to inherit from the class `EventPayload`. The following events are being specified within the Gui project itself:

- `AddPcbObjectsEvent` - this event occurs when the importing of objects from the **PCB**-Investigator API has been finished
- `ChangeExportMenuItemEnabledStateEvent` - will be created to toggle the enabled states of menu items for exporting data into file
- `CloseAllSettingsEvent` - event which will be called if the settings page will be closed
- `ComponentsImportFinishedEvent` - that event will be generated if the import of **JSON** related data was finished
- `HandleDropEvent` - an event which will be called if something was dropped my mouse into the tool
- `ResetLayersSelectionEvent` - is being created to trigger the reset of all layers which will be shown within the view section
- `ResetViewEvent` - if the whole view needs to be reset then this event is being called
- `SelectBomFinishEvent` - after a bill of material file was successfully selected then this event will be created
- `SendLayersEvent` - to send all created layers after the read out from **PCB**-Investigator API to the view this event is needed
- `SetBusyEvent` - for being able to set the state of the is busy animation of the application this event could be used
- `SetCoverageValuesEvent` - in order to update all test coverage determined values of objects this event needs to be used
- `ShowObjectsEvent` - showing of different kind of objects which are located in specific layers could be performed by using that event

- UpdateBomDataEvent - updating all bill of materials related data of objects could be performed by using this event

```
1 eventService.Subscribe<SetBusyEvent>(HandleIsBusyEvent ,
    ThreadOption.UIThread);
```

Listing 5.3: Subscribing to an event

As presented in 5.3 there further could be specified some thread options to be considered at subscribing to an event. In most cases there was just the thread option UIThread necessary.

```
1 eventService.Publish(new SetBusyEvent(true));
```

Listing 5.4: Publishing of an event

As shown in 5.4 publishing could be realized just by passing a constructor of the event of interest containing all necessary parameters for it.

5.5.3. Helper

Now for being able to perform different kind of additional work for finishing tasks there was the need of providing some helper classes which could be used for that:

- AttributeList - a class which just contains all necessary identifier for the different kind of objects received by the PCB-Investigator API
- BomSettings - this class holds all bill of material related settings (separator, column number of ref and column number of value)
- CustomCollectionException - that customized exception class was necessary for avoiding the IDE to gripe about the catching of general exception types
- DropFilesBehaviorExtension - within that class all the behavior regarding the drop mechanism into the application are defined
- DummyDataHandler - to use a dummy data handler for testing purposes this class has been provided
- LogMessage - for being able to handle a logging message and it's logging category this class was used
- Manifest - the class which contains all necessary properties of working with the manifest project mode

- PositionHelper - to determine all necessary positions of the points of a connection line between several objects this class was necessary to be introduced
- StorageColor - as it was necessary to work with serializable color information that class was needed
- ViewModelFactory - another **factory pattern** using class for creation of wrapping viewModels for rendering in dependency on the specific object type

```

1  private static void OnPropChanged(DependencyObject d,
    DependencyPropertyChangedEventArgs e)
2  {
3      if (!(d is FrameworkElement fe))
4      {
5          return;
6      }
7
8      if ((bool)e.NewValue)
9      {
10         fe.AllowDrop = true;
11         fe.Drop += OnDrop;
12         fe.PreviewDragOver += OnPreviewDragOver;
13     }
14     else
15     {
16         fe.AllowDrop = false;
17         fe.Drop -= OnDrop;
18         fe.PreviewDragOver -= OnPreviewDragOver;
19     }
20 }

```

Listing 5.5: OnPropChanged

As shown in 5.5 within the function for registering the drop mechanism there will be added specific methods to be called when the Drop or PreviewDragOver event occur of any rendered object.

5.5.4. Interfaces

To be able to work as flexible as possible with the used logic of course there was the need of providing several interfaces which could be used by different implementations to be registered:

- IBomDataModel - interface for storing all bill of materials related data and modifying it

- IBomHandler - the interface for handling all bill of materials related actions to be performed
- IComponentViewModel - this interface is being used for defining the viewModel component related actions as toggling the visibility or showing all connections of it
- IDataSourceProvider - that interface is being created for defining the different data sources of project loading
- IDropHandler - an interface for defining the drop event handling
- IFilesDropped - one interface for specifying the action to be performed on multiple files dropped
- IManifestHandler - specific interface which is responsible for all actions regarding the manifest project handling
- IProjectFileHandler - how the project file handling is defined in general could be set by using this interface
- IProjectHandler - for being able to define the data source flow regarding the import and export of project related data could be found here
- IProjectLoadHandler - all flow handling related functions regarding the import and export functionality of the project could be used by this interface
- IResultModel - as there should be the possibility of holding all received data by the **PCB**-Investigator within one model there was this interface introduced
- ISettingsData - to be able to work with all settings in generic way there was this interface provided
- ISettingsHandler - importing and exporting of all settings data could be specified by using this interface
- ISettingsStorageManager - saving and getting all settings content could be defined by implementing that interface
- ISettingsStorageModel - model interface for defining the serialized content of all settings could be found within here

```

1 public interface IComponentViewModel
2 {
3     Task ToggleCompleteVisibility(Type typeToHide);
4
5     Task ToggleShowConnections(ViewModelPCBComponentBase comp,
        bool shouldShow);

```

```

6
7     Task ShowLayerObjects(IList<string> layersToShow);
8 }

```

Listing 5.6: IComponentViewModel

As shown in 5.6 there were Task objects used for being able to work with the async await operations to make sure not to block the Gui while operation is running[22].

5.5.5. Implementations

As there were previously many interfaces introduced of course there should be as many implementations for usage provided:

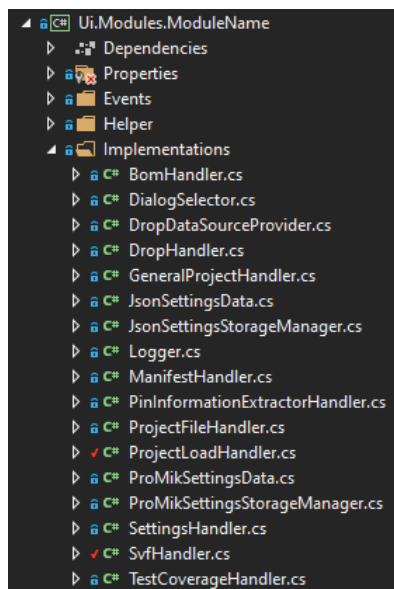


Figure 5.5.: All implementations within the Ui.Modules.ModuleName project

As displayed in 5.5 there are more implementations available as predefined before within the Interfaces section. That comes from the fact that there are much more interfaces available in other projects which needs to be considered on that place as the main sequences are all being triggered from that project.

```

1 public bool OpenGenericDialog(DialogType dialogType, string
   title, string errorMessage, out string selectedTarget, string
   filter = "")
2 {
3     string content = string.Empty;

```

```

4      if (dialogType == DialogType.OPENFILE)
5      {
6          VistaOpenFileDialog openDialog = new
              VistaOpenFileDialog() { Title = title, Filter =
                  filter };
7          openDialog.ShowDialog();
8          content = openDialog.FileName;
9      }
10     else if (dialogType == DialogType.SAVEFILE)
11     {
12         VistaSaveFileDialog saveDialog = new
              VistaSaveFileDialog
13         {
14             Title = title,
15             Filter = filter,
16         };
17         saveDialog.ShowDialog();
18         content = saveDialog.FileName;
19     }
20     ...

```

Listing 5.7: OpenFileDialog method of IDialogSelector implementation

As stated in 5.7 there was used the VistaOpenFileDialog and VistaSaveFileDialog from an external library used by the nuget package manager for handling the path selection within a dialog. If needed just by using another implementation of this interface the complete handling and used packages for it could be easily changed.

5.5.6. Model

To be able to hold different kind of data project wide and access it by dependency injection in comfortable way there are some models been defined:

- BomDataModel - the model for holding all bill of material related data
- ResultModel - this model saves all object related results after requesting the PCB-Investigator API
- SettingsStorageModel - that model is responsible for having all setting related data available
- TestCoverageDataModel - a model for saving all test coverage related objects of interest for determination of it

```
1 public class ResultModel : IResultModel
2 {
3     public IParsedResult Result { get; set; }
4
5     public bool CheckIfValuesAreBeingUsed()
6     {
7         foreach (var comp in Result.Components)
8         {
9             if
10                (!string.IsNullOrEmpty(comp.FunctionalAttributes.Value))
11            {
12                return true;
13            }
14
15            return false;
16        }
17 }
```

Listing 5.8: ResultModel

As stated in 5.8 there was additionally to the property a helperfunction defined for being able to determine whether all values from bill of materials import should be considered or not. In dependency on that check later the logic is able to highlight all objects which have no valid value received.

5.5.7. ViewModels

Within that folder are all necessary viewModels for that project are defined:

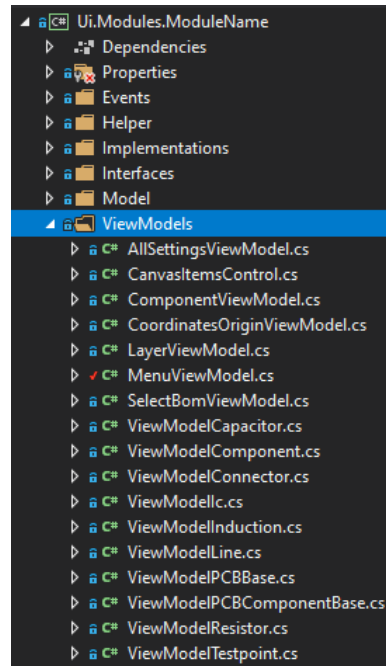


Figure 5.6.: All viewModels within the Ui.Modules.ModuleName project

As shown in 5.6 there were used the following main types of viewModels:

- Gui related viewModels which are necessary to provide functionality for visible and click able content by the user within the main view
- a helper viewModel for representation of the origin of the coordinate system
- object related viewModels for representation of all available object types from the read out ODB++ project (resistor, capacitor, induction...)

5.5.8. Views

That folder contains all views for representation of specific regions of the mainView or components to be rendered:

- AllSettingsView - this view contains all settings of the software by using the ProMik.UI.WPF.SettingsPanel which was included as a nuget package
- ComponentView - that view contains all necessary appearance settings for displaying all objects to be rendered within the mainView region
- CoordinatesOrigin - within that view just the representation form of the origin of the coordinate system is defined

- LayerView - the view for showing all received layers from PCB-Investigator API which is been included in a region of the mainView
- MenuView - all menu items which are accessible by the user are defined in that view
- PCBComponentBaseDataTemplate - to render any received object from a ODB++ project this template will be used
- PCBLineDataTemplate - for being able to show connections between objects that view will be considered as template for drawing the lines
- PCBTstpointDataTemplate - as the testpoint itself is not a rectangle but a circle it has it's own template defined in that view
- SelectBomView - to make sure to display a window containing all bill of material related settings before importing any comma separated values file this view was provided

```

1  ...
2  <DataTemplate DataType="{x:Type if:ViewModelLine}">
3      <Canvas Visibility="{Binding LineVisibility}">
4          <Canvas.ToolTip>
5              <ToolTip Visibility="Visible"
6                  Content="{Binding ObjectNames}"
7                  Placement="MousePoint" />
8          </Canvas.ToolTip>
9          <Canvas.ContextMenu>
10             <ContextMenu>
11                 <MenuItem Header="Hide"
12                     Command="{Binding Hide}" />
13             </ContextMenu>
14         </Canvas.ContextMenu>
15         <Path Data="{Binding Points}" Opacity="{Binding
16             Opacity}" StrokeThickness="3"
17             Stroke="{Binding LineColor}"/>
18         <Label Content="{Binding NetName}"
19             Opacity="{Binding Opacity}">
20             <Label.RenderTransform>
21                 <TranslateTransform X="{Binding
22                     XPositionNetName}" Y="{Binding
23                     YPositionNetName}"/>
24             </Label.RenderTransform>

```

```
17         </Label>
18     </Canvas>
19 </DataTemplate>
20 ...
```

Listing 5.9: PCBLineDataTemplate

As shown in 5.9 there could be seen that to ensure a loose coupling implementation there are being used many bindings to values and commands. Further it will be clear that this template should only be used if the representative object type is a `ViewModelLine`. By using the command `Hide` it will be possible to hide any rendered line completely by choosing the action within the context menu of the right mouse button click.

5.6. Test coverage determination of loaded project

The **TestCoverage** project which was implemented using the .NET Standard 2.1 framework is responsible for determining all test related objects and different kind of test coverage results to be displayed within the application as plain text. It consists of the following sub folders and content:

- Events - here are the test coverage related events are placed which could be used for triggering some actions
- Helper - that folder contains some helper classes which are necessary to work with the other interfaces of this project in a comfortable way
- Interfaces - this folder consists of all interfaces which are being used by other projects to perform test coverage related actions and could be easily extended by providing new implementations for them
- Implementations - within that folder are actually just boundary scan related implementations of the interfaces to be used

Actually there is just been provided the possibility of determining boundary scan related values according the following sequence:

1. Define the settings for general identifying the ICs within the tool
2. Define the test coverage related settings within the tool
 - ground nets related identifier
 - ground nets related blacklist

- power nets related identifier
 - power nets related blacklist
 - jtag nets related identifier
 - jtag nets related blacklist
3. Load a **ODB++** project into the tool
 4. Perform the function **Determine all boundary scan objects** by menu item
 5. Perform the test coverage determination for at least one of the following values by clicking the according menu item:
 - Determine test coverage for JTAGs - the amount of all **JTAG** devices / amount of total pins of all **JTAG** devices
 - Determine test coverage for all PullUps - the amount of totally connected power nets of all **JTAG** devices / amount of total pins of all **JTAG** devices
 - Determine test coverage for all PullDowns - the amount of totally connected ground nets of all **JTAG** devices / amount of total pins of all **JTAG** devices
 - Determine test coverage for all PullUps and PullDowns - (the amount of totally connected ground nets + power nets of all **JTAG** devices) / amount of total pins of all **JTAG** devices
 - Determine test coverage for all other components - the amount of totally connected nets which are neither ground nor power nets of all **JTAG** devices / amount of total pins of all **JTAG** devices
 - Determine complete test coverage of boundary scan - (the amount of totally connected nets which are neither ground nor power nets + connected ground + connected power nets of all **JTAG** devices) / amount of total pins of all **JTAG** devices
 6. Finally the determined value will be updated and visible as textual representation at the bottom of the tool

Next there are some code snippets presented to have a closer look into the implementation of background logic:

```

1 public interface ITestCoverageDeterminer
2 {
3     bool
4         IsTestResultAvailableForComponentsAndType(IPCBComponent
5             comp, IPCBComponent second);
6
7     void ClearAllObjects();
8
9     Task<TestCoverageItems> GetTestCoverage(TestCoverageItems
10         items, TestCoverageType type);
11
12     TestCoverageItems DetermineTestCoverageRelatedObjects(
13         IList<INetComponent> nets,
14         IdentifierBlacklistContainer content,
15         TestCoverageItems items);
16 }

```

Listing 5.10: ITestCoverageDeterminer

As stated in 5.10 it is clear that for being able to perform any `GetTestCoverage` function there will be used a enum type of `TestCoverageType` which could be easily extended for future purposes. Next in dependency on two received objects its possible to detect whether any test coverage is available or not to decide later on whether to draw the connection lines between objects in green color or just black.

```

1 public TestCoverageItems DetermineTestCoverageRelatedObjects(
2     IList<INetComponent> nets,
3     IdentifierBlacklistContainer content,
4     TestCoverageItems items)
5 {
6     if (nets != null && nets.Count > 0)
7     {
8         DefineGndNets(nets, content.GndNetIdentifier,
9             content.GndNetBlacklist);
10        DefinePowerNets(nets, content.PowerNetIdentifier,
11            content.PowerNetBlacklist);
12        DefineJtagNets(nets, content.JTAGNetIdentifier,
13            content.JTAGNetBlacklist);
14        Dictionary<TestCoverageObject, IList<IPCBComponent>>
15            mapObjects = new Dictionary<TestCoverageObject,
16                IList<IPCBComponent>>();
17        testCoverageDataModel.Ics = DefineIcs();
18        testCoverageDataModel.PullUps =
19            DefinePullUpResistors();
20    }
21 }

```

```

14         testCoverageDataModel.PullDowns =
            DefinePullDownResistors();
15         mapObjects.Add(TestCoverageObject.PULLUP,
            testCoverageDataModel.PullUps);
16         mapObjects.Add(TestCoverageObject.PULLDOWN,
            testCoverageDataModel.PullDowns);
17         mapObjects.Add(TestCoverageObject.JTAG,
            testCoverageDataModel.Ics);
18         mapObjects.Add(TestCoverageObject.OTHERS,
            DefineOthers());
19         LogTestCoverageObjects(mapObjects);
20         eventService.Publish(new
            AddTestCoverageObjectsEvent(mapObjects));
21         eventService.Publish(new
22             TestCoverageForDeterminingObjectsPerformedEvent(
23                 TESTCOVERAGEOBJECTS.ToList()));
24         items.ItemTestCoverageObjects =
            TestCoverageBoundaryScanDeterminer.ITEMSELECTED;
25         if (mapObjects.Count > 0)
26         {
27             items.TestCoverageMenuItemsEnabled = true;
28         }
29
30         CheckDoubleMatching(testCoverageDataModel.PullDowns,
            testCoverageDataModel.PullUps);
31         LogResults();
32     }
33     else
34     {
35         items.ItemTestCoverageObjects =
            TestCoverageBoundaryScanDeterminer.ITEMNOTSELECTED;
36     }
37
38     return items;
39 }

```

Listing 5.11: DetermineTestCoverageRelatedObjects
TestCoverageBoundaryScanDeterminer

of

As shown in 5.11 there are first of all within the function of defining all test coverage related objects, the nets are being defined before then all test coverage related objects are being determined. Finally a dictionary containing all objects in relation to a test coverage object type is being filled and send by an event to be filled within the LayerRegion for providing the user the possibility of selecting them to be showed. As a feature there will be checked additionally if some components were detected being more than one

test coverage related object. If this is the case then some logging output as a warning category will be performed to inform the user about settings to be adapted in case of improving the identifier or blacklist of specific object types.

5.7. Pin information extraction

To be able to extract all pin information of all **JTAG** related devices of any **ODB++** project into separate files to be created there is the project **PinInformationExtractor** available which was created using the .NET standard 2.1 framework as well. It contains the following sub folders with the specific content:

- Interfaces - this folder contains all interfaces which could be used for performing actions of this project
- Implementations - within that folder are the implementation of the interface could be found
- Helper - that folder consists of some helper classes which are being used within the implementation class of this project

```

1 public interface IPinInformationExtractor
2 {
3     List<JtagConnectionInfo> CreatePinInformationFile(string
4         filePath, IList<IPCBCComponent> ics, IList<IPCBCComponent>
5         pullUps, IList<IPCBCComponent> pullDowns, string
6         tdiIdentifier = "tdi", string tdoIdentifier = "tdo",
7         string tckIdentifier = "tck", string tmsIdentifier
8         = "tms");
9     List<string> GetAllGroundPins(List<PinConnectionInfo> pins);
10    List<string> GetAllPowerPins(List<PinConnectionInfo> pins);
11    List<JtagConnectionInfo> GetAllPinInformation(IList
12        <IPCBCComponent> ics, IList<IPCBCComponent> pullUps, IList
13        <IPCBCComponent> pullDowns, string tdiIdentifier = "tdi",
14        string tdoIdentifier = "tdo", string tckIdentifier
15        = "tck",
16        string tmsIdentifier = "tms");
17 }

```

Listing 5.12: IPinInformationExtractor

As shown in 5.12 there are much of the passed parameters for creation of the pin information file set per default to a specific value which is being used by the main routine actually. By preallocation of parameters the user does not have to exclusively define

them manually when calling that specific method which leads to a more comfortable way of working.

5.8. SVF file generator

Now for being able to create **SVF** files with all specific commands needed for being able to be played with a programmer by ProMik there is the **SVFHelper** project been provided using as well the .NET Standard 2.1 framework.

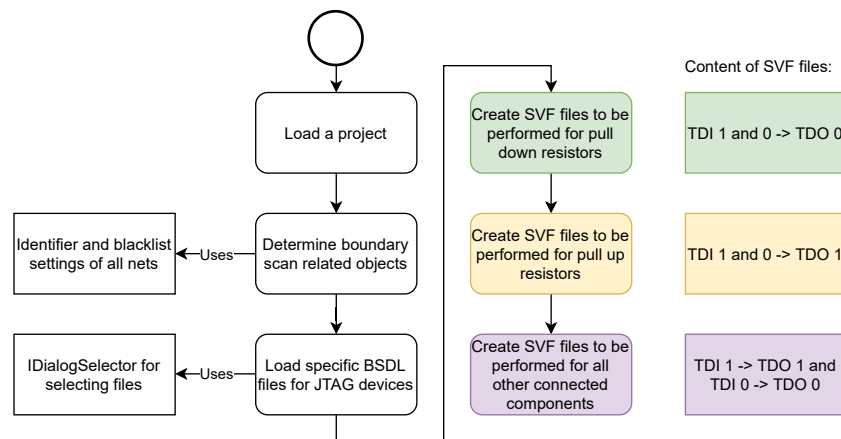


Figure 5.7.: The program schedule for creation of **SVF** files

As shown within 5.7 there is the need of selecting manually all **BSDL** files for all **JTAG** devices in order to get all pin information regarding their boundary scan compatibility and position within the **SVF** test vector to be generated. To provide all necessary functionality to be able to perform that steps the project consists of the following sub folders and classes:

- Interfaces - within that folder all interfaces which are available located at
- Implementations - all implementations of the interfaces are placed within that folder
- Helper - in that folder an exception helper class could be found which was necessary to avoid the **IDE** complaining about catching an general exception

```

1 public class PullDownSvfWriter : ISvfWriter
2 {
3     private readonly ISVFHelper svfHelper;
4 
```

```

5      public PullDownSvfWriter(ISVFHelper svfHelper)
6      {
7          this.svfHelper = svfHelper;
8      }
9
10     public List<ISvfData> GetSVFFiles(string basePath, string
11         jtagName, List<string> pinLabel, IBSDLOutput bsdlOutput,
12         byte[] defaultVector)
13     {
14         List<ISvfData> data = new List<ISvfData>();
15         foreach (var pin in pinLabel)
16         {
17             data.Add(svfHelper.GetSvfContent(basePath,
18                 jtagName,
19                 pin, bsdlOutput, false, false, defaultVector));
20         }
21         foreach (var pin in pinLabel)
22         {
23             data.Add(svfHelper.GetSvfContent(basePath,
24                 jtagName,
25                 pin, bsdlOutput, true, false, defaultVector));
26         }
27         return data;
28     }
29 }

```

Listing 5.13: PullDownSvfWriter

As shown in 5.13 it will be clear that there is being a ISvfData model used for collecting all necessary information about one SVF file. Next as the logic for the pull down resistors is being shown it could be seen that passing on the one side false and on the other side true within the GetSvfContent function means that this is the TDI value to be used.

```

1  private string GenerateNewVector(string pinLabel, bool value,
2      Dictionary<int, IBoundaryCell> boundaryCells, ref int
3      position, byte[] defaultVector)
4  {
5      var index = 0;
6      foreach (var cell in boundaryCells)
7      {
8          if (cell.Value.Port.Equals(pinLabel))
9          {
10             index = cell.Key;
11             position = index;

```

```

10             break;
11         }
12     }
13
14     if (index == 0)
15     {
16         return string.Empty;
17     }
18
19     var vectorArray = new BitArray(defaultVector);
20     try
21     {
22         vectorArray.Set(index, value);
23     }
24     catch (ArgumentOutOfRangeException e)
25     {
26         logger.LogMessage(e.Message, LogCategory.ERROR);
27     }
28
29     byte[] resultBytes = new byte[((vectorArray.Length - 1) /
30         8) + 1];
31     vectorArray.CopyTo(resultBytes, 0);
32     Array.Reverse(resultBytes, 0, resultBytes.Length);
33     return ByteArrayToString(resultBytes);
34 }

```

Listing 5.14: GenerateNewVector of SVFHelper

As shown in 5.14 there will be first searched after the pin label within the dictionary received by the **BSDL** file. If a valid index was found then the default vector will be converted into a bit array and the specific index of the pin will be set to the requested value. Finally the bit array will be converted back to a hexadecimal representation. If there was no index found at all just an empty string will be returned to trigger the logging of missing pin information within the **BSDL** file. The default vector must be read out directly before creation of **SVF** files and contains all actual states of pins in a hexadecimal representation.

5.9. Unit tests

As unit tests are highly recommended for guaranteeing a high quality of software logic[23] of course this solution also contains some unit tests which are separated in the **Test-DotNetCore** project which was created using the .NET Core 3.1 framework and the

TestDotNetFramework project which was implemented using the .NET Framework 4.72.

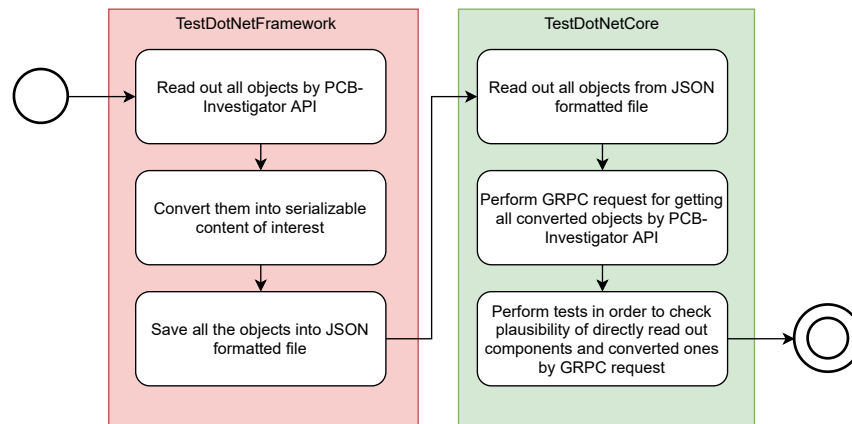


Figure 5.8.: The sequence for a complete test of parsing validation of objects received by **PCB**-Investigator API

As shown in 5.8 there was the need of exporting test related data into a local file which could be read in later for finalizing the tests as the usage of two different **GRPC** related frameworks of project creation made it impossible to test everything just within one test. For that purpose there was introduced the helper class `PcbTestObject` to provide the following information:

- Name - the name of a component
- TotalAmountOfPins - the total amount of connected pins
- Pins - a list containing `PinTestObjects`

The included `PinTestObject` contains the following information of interest:

- PinNumber - the pin number
- Nets - a list containing all net names which are connected to this pin

```

1 [Fact]
2 public void TestParsingFromApi()
3 {
4     IGrpcClientParserHandler grpcHandler = new
5         GrpcClientParserHandler(new DebugLogger());
6     List<PcbTestObject> resultData = GetSerializedData(FILEPATH);
7     Assert.NotNull(resultData);
8     Assert.True(resultData.Count > 0);
  
```

```
8      var result =
          grpcHandler.GetParsedObjectsFromGrpc(PATHTOODB,
              "-1").Result;
9      Assert.NotNull(result);
10     Assert.True(result.Components.Count > 0);
11     Assert.True(result.Nets.Count > 0);
12     TestContent(resultData, result);
13     Debug.WriteLine("Tests finished successfully!");
14 }
```

Listing 5.15: TestParsingFromApi from TestDotNetCore

As stated in 5.15 there first of all will be asserted that the received objects by the **GRPC** API is not null and the read out content from the **JSON** formatted file contains some objects. Finally there will be called the `TestContent` function to make sure that all objects equals the necessary information. Further it can be seen that for testing purposes a `DebugLogger` was generated which implements the `ILogger` interface in that way that all messages are just being forwarded to the debug line as no other objects are available running this sequence. The `-1` parameter within the `GetParsedObjectsFromGrpc` method tells the API to check for all available steps within the **ODB++** project as there is the possibility to define specific ones to be considered by using a semicolon separated string.

Chapter 6.

Analysis

- 6.1. Performance of loading ODB++ projects by using the PCBInvestigator API
- 6.2. Performance of loading projects by importing JSON formatted files
- 6.3. Performance of exporting projects into JSON format
- 6.4. Performance of showing connections of components
- 6.5. Performance of determination of test coverage values

Chapter 7.

Conclusion

Chapter 8.

Thanksgiving

Within this small chapter I would like to thank a couple of people who made this work possible by supporting me in many different areas of my life.

First of all I want to thank my parents **Gabriela** and **Nicolae Matis** who have born me and supported me in every single minute of my life. I know I unfortunately didn't always made you proud of my actions but I hope that I am still working on it successfully by milestones like this one. Mum I really hope for you that you will solve your physical constraints successfully and dad I hope that you stay as young as you are for a really long time

Afterward I would like to thank my brother **Nik Matis** and my sister **Michelle Matis** who have supported me in that way as a brother just could expect it to be perfect. Michelle I wish you all the best with your study and Nik I hope you will find your destiny in choosing a new job soon.

And finally I would like to thank a couple of my best friends who have supported me in investing much free time with myself and were successfully responsible for being able to forget all the stress about the work and enjoy my life.

Andreas Korkenländer I already know you now about for 20 years and really hope that this will continue until my death. I wish you all the best with your new job and your intention of traveling around the world.

Dominik Müller you were my first best friend after I moved to Nürnberg and I really hope that connection persists for a longer time although you are sometimes really a hardheaded person :) I wish you all the best with your new challenge of doing a new apprenticeship.

Rene Biedermann you are one of that guys I'm able to speak about philosophic and esoteric topics and of course medical topics about my own health in a detailed way as I'm really impressed of it. I wish you all the best in accomplishing your exam for being

an alternative practitioner and I really think that this will satisfy your life in complete way.

Willi Fritz I would really like to thank you especially for the last year where you permitted me to do my training within your own flat and visited me almost every weekend for just hanging around and playing PlayStation 4 or for cooking some real tasty meals. I wish you all the best at finishing your tax consultant state examination and am really looking forward of traveling around the world with your person soon.

Appendix A.

Appendix

List of Figures

3.1. ODB++ project structure[2]	6
3.2. PCB-Investigator overview	7
3.3. JTAG interface in detail[4]	8
3.4. State machine of the TAP controller[6]	9
3.5. MVVM structure[13]	12
4.1. Client-server structure of the planned software	17
4.2. Frameworks used within the project	18
4.3. Class diagram of the PCB-Investigator objects of interest	19
4.4. Class structure after conversion	20
4.5. Physical object dependencies	21
5.1. Project overview	23
5.2. Program schedule of the Grpc server project	26
5.3. IGrpcClientParserHandler class diagram	28
5.4. The mainView with its regions	30
5.5. All implementations within the Ui.Modules.ModuleName project	35
5.6. All viewModels within the Ui.Modules.ModuleName project	38
5.7. The program schedule for creation of SVF files	45
5.8. The sequence for a complete test of parsing validation of objects received by PCB-Investigator API	48

List of Tables

4.1. Programming languages comparison	16
4.2. IDE comparison	17
4.3. Framework comparison	18

List of Listings

3.1.	JSON formatted content	13
3.2.	Interface definition of GRPC exchange	14
5.1.	GetPinsGrpc	27
5.2.	GetComponentByType	28
5.3.	Subscribing to an event	32
5.4.	Publishing of an event	32
5.5.	OnPropChanged	33
5.6.	IComponentViewModel	34
5.7.	OpenGenericDialog method of IDialogSelector implementation	35
5.8.	ResultModel	37
5.9.	PCBLineDataTemplate	39
5.10.	ITestCoverageDeterminer	41
5.11.	DetermineTestCoverageRelatedObjects of TestCoverageBoundaryScanDeterminer	42
5.12.	IPinInformationExtractor	44
5.13.	PullDownSvfWriter	45
5.14.	GenerateNewVector of SVFHelper	46
5.15.	TestParsingFromApi from TestDotNetCore	48

Bibliography

- [1] ODB++Design, “Why odb++design - odb++design,” 5/18/2020. [Online]. Available: <https://odbplusplus.com/design/why-odb-d/>
- [2] “What is odb++,” 2/5/2019. [Online]. Available: https://www.artwork.com/odb++/odb++_overview.htm
- [3] “Pcb-investigator,” 4/7/2021. [Online]. Available: <https://www.pcb-investigator.com/de/functions>
- [4] XJTAG, “Was ist jtag und wie kann ich es nutzen? - xjtag tutorial,” 8/27/2020. [Online]. Available: <https://www.xjtag.com/de/about-jtag/what-is-jtag/>
- [5] “Joint test action group – wikipedia,” 3/25/2021. [Online]. Available: https://de.wikipedia.org/wiki/Joint_Test_Action_Group
- [6] XJTAG, “Technischer leitfaden für jtag boundary-scan - xjtag tutorial,” 13.03.2020. [Online]. Available: <https://www.xjtag.com/de/about-jtag/jtag-a-technical-overview/>
- [7] Harry Bleeker, *Boundary-Scan Test: A Practical Approach*. Springer Science+Business Media Dordrecht, 1993.
- [8] R.C. Cofer and Benjamin F. Harding, *Rapid System Prototyping with FPGAs*. Elsevier Inc., 2006.
- [9] “Boundary scan description language – wikipedia,” 3/25/2021. [Online]. Available: https://de.wikipedia.org/wiki/Boundary_Scan_Description_Language
- [10] “Serial vector format – wikipedia,” 3/25/2021. [Online]. Available: https://de.wikipedia.org/wiki/Serial_Vector_Format
- [11] TEXAS INSTRUMENTS, ASSET InterTech, Inc., *SVF SERIAL VECTOR FORMAT SPECIFICATION JTAGBOUNDARY SCAN*, 2013.
- [12] Norbert Eder, “Mvvm: Das viewmodel,” 2010. [Online]. Available: <https://www.norberteder.com/MVVM-Das-ViewModel/>
- [13] “Model–view–viewmodel,” 2021. [Online]. Available: <https://en.wikipedia.org/w/index.php?title=Modelviewviewmodel&oldid=1012221294>

- [14] “Introduction to prism | prism,” 4/5/2021. [Online]. Available: <https://prismlibrary.com/docs/>
- [15] S. Augsten, “Was ist dependency injection?” 4/9/2021. [Online]. Available: <https://www.dev-insider.de/was-ist-dependency-injection-a-814452/>
- [16] “Components - material design,” 1/1/1980. [Online]. Available: <https://material.io/components>
- [17] “Json,” 29.03.2021. [Online]. Available: <https://www.json.org/json-en.html>
- [18] gRPC, “<meta property,” 4/8/2021. [Online]. Available: <https://grpc.io/>
- [19] “Pcb-investigator interface documentation - table of content,” 4/7/2021. [Online]. Available: <https://manual.pcb-investigator.com/InterfaceDocumentation/index.php>
- [20] JAXenter, “Mit factory-pattern designprobleme lösen - jaxenter,” 2012. [Online]. Available: <https://jaxenter.de/mit-factory-pattern-designprobleme-loesen-5040>
- [21] “Strategie (entwurfsmuster),” 2021. [Online]. Available: [https://de.wikipedia.org/w/index.php?title=Strategie_\(Entwurfsmuster\)&oldid=207453256](https://de.wikipedia.org/w/index.php?title=Strategie_(Entwurfsmuster)&oldid=207453256)
- [22] BillWagner, “Asynchrone programmierung in c#,” 4/12/2021. [Online]. Available: <https://docs.microsoft.com/de-de/dotnet/csharp/programming-guide/concepts/async/>
- [23] Udemy, “Unit testing in .net,” 4/12/2021. [Online]. Available: https://www.udemy.com/course/unit-testing-in-net/?utm_source=adwords-learn&utm_medium=udemyads&utm_campaign=INTL-AW-PROS-ALL-DE-DSA-DE-GER_.ci_.sl_GER_.vi_ALL_.sd_All_.la_DE_.&utm_content=deal4584&utm_term=_.ag_61564678013_.ad_372874095721_.de_c_.dm_.pl_.ti_dsa-560803889966_.li_9060719_.pd_.&gclid=Cj0KCQjw38-DBhDpARIsADJ3kjk98ZhmUUxudjCct9kpdSkHacXHamNdZgQfViizecgd0P8L8ooYspwcB